Section 4.1 — Database Taxonomy & Schema Selection

Enhanced Study Notes | 5 Files in Series (4.1–4.5)

> ***Study Directive (5 files):*** *This is file 1 of 5 in the Section 4 database series. This section is* ***architecture-heavy and comparison-heavy****. Give extra weight to before/after schema decisions, named-system comparisons, and decision frameworks for choosing the right database for a given workload.*

Executive Overview

Database selection is one of the highest-leverage architectural decisions. The wrong choice cannot easily be undone — migration is painful, expensive, and risky. This section equips you to reason about ACID guarantees, NoSQL trade-offs, and schema patterns with enough depth to justify choices in an interview or production context.

---

1. Relational Databases (RDBMS)

1.1 ACID — What Each Letter Actually Means

Engineers often recite ACID without understanding the failure modes each property prevents.

```
ATOMICITY — protects against partial writes

  Scenario without atomicity:
    Transfer $100 from Alice → Bob:
    Step 1: Debit Alice  -$100  ← power cut here
    Step 2: Credit Bob   +$100  ← never executes
    Result: $100 vanishes from the system

  With atomicity:
    BEGIN TRANSACTION;
    UPDATE accounts SET balance = balance - 100 WHERE id = 'alice';
    UPDATE accounts SET balance = balance + 100 WHERE id = 'bob';
```

```
    COMMIT;  -- both happen, or neither happens

CONSISTENCY — protects against constraint violations

  Scenario: Foreign key constraint
    INSERT INTO orders (user_id=999) -- user 999 doesn't exist
    Without consistency: orphaned order row
    With consistency: constraint fires, INSERT rejected

  Other consistency enforcers:
    CHECK constraints: balance >= 0
    UNIQUE constraints: one email per account
    Triggers: derived data maintenance

ISOLATION — protects against interference between concurrent transactions

  Four isolation levels (weakest → strongest):
    READ UNCOMMITTED → READ COMMITTED → REPEATABLE READ → SERIALIZABLE
  (Full isolation anomaly table in Section 4.3)

DURABILITY — protects against data loss after COMMIT

  Mechanism: Write-Ahead Log (WAL)
    1. Write intent to WAL (sequential, fast)
    2. Acknowledge COMMIT to client
    3. Apply WAL to data pages (background)
    4. Crash recovery: replay WAL from last checkpoint

  Trade-off — delayed durability:
    PostgreSQL: synchronous_commit = off
    Risk: up to 200ms of committed transactions lost on crash
    Gain: 3-5x write throughput improvement
    Use case: Metrics, logs — acceptable loss window
```

1.2 Normalization — When to Normalize and When Not To

```
FIRST NORMAL FORM (1NF) — Eliminate repeating groups

  BEFORE (violates 1NF):
    orders: { id=1, products="pen,pencil,ruler", prices="1.00,0.50,2.00" }
    Problem: Cannot query "all orders containing a pen" without string parsing

  AFTER:
    orders: { id=1, customer_id=42 }
    order_items: { order_id=1, product="pen", price=1.00 }
                 { order_id=1, product="pencil", price=0.50 }

SECOND NORMAL FORM (2NF) — No partial dependency on composite key

  BEFORE (violates 2NF):
    order_items: { order_id, product_id, product_name, quantity }
    ↑ product_name depends only on product_id, not the full composite key
```

```
  AFTER:
    order_items: { order_id, product_id, quantity }
    products:    { product_id, product_name }

THIRD NORMAL FORM (3NF) — No transitive dependency

  BEFORE (violates 3NF):
    orders: { id, customer_id, customer_city, customer_zip }
    ↑ customer_city depends on customer_zip (not on order id)

  AFTER:
    orders:    { id, customer_id }
    customers: { id, city, zip }

WHEN TO VIOLATE NORMALIZATION INTENTIONALLY:
  - Reporting tables: denormalize for query speed
  - Feed precomputation: embed post data in user_feeds
  - Hot-path reads: avoid joins on critical paths
  Rule: Normalize for writes. Denormalize (selectively) for reads.
```

---

2. NoSQL Storage Models — Full Comparison

2.1 Key-Value Stores

When to use: Simple lookup by a single key, caching, sessions, leaderboards. **When NOT to use:** Complex queries, relationships, multi-key transactions.

```
Redis data structures and their use cases:

  String   → Simple cache entries, counters, distributed locks
             SET session:abc123 "{user_id:42}" EX 3600
             INCR page_views:homepage  ← atomic counter

  Hash     → Object storage (avoid JSON parse overhead)
             HSET user:42 name "Alice" email "alice@ex.com" age 30
             HGET user:42 name  ← fetch single field

  List     → Message queues, recent activity feeds
             LPUSH notifications:42 "New follower"
             LRANGE notifications:42 0 9  ← latest 10

  Set      → Unique members, intersection (mutual friends)
             SADD followers:alice bob charlie dave
             SINTER followers:alice followers:bob  ← mutual followers

  Sorted Set → Leaderboards, priority queues
             ZADD leaderboard 1500 "alice" 1200 "bob"
             ZREVRANGE leaderboard 0 9  ← top 10
```

```
  Streams  → Event sourcing, pub/sub with persistence
             XADD events * type "purchase" amount 99.99
```

DynamoDB — when partition key design is critical:

```
Good partition key: high cardinality, uniform access
  user_id (millions of unique values) ← good
  status  ("active"/"inactive")       ← bad: hot partition!

Hot partition problem:
  Imagine: partition_key = "US" (most users are US)
  All US traffic hits 1 partition → throughput exceeded → throttling

  Fix 1: Add random suffix → "US_1", "US_2", "US_7" (scatter-gather reads)
  Fix 2: Use composite key → user_id + date (high cardinality)
  Fix 3: Use GSI with better partition key for alternate query

Global Secondary Index (GSI) — enables alternate access patterns:
  Table primary key: user_id
  GSI partition key: email  ← look up user by email without full scan
  Consistency: eventually consistent (GSI replication async)
  Cost: every write duplicated to GSI storage
```

2.2 Document Stores

When to use: Hierarchical/nested data, flexible schema, catalog-style reads. **When NOT to use:** Strong relational integrity, complex cross-document queries.

```
MongoDB document model:

{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "sku": "LAPTOP-X1",
  "name": "ProBook Laptop",
  "category": "electronics",
  "specs": {                    ← nested object (no JOIN needed)
    "ram_gb": 16,
    "cpu": "Intel i7-12th",
    "storage_gb": 512
  },
  "variants": [                 ← embedded array (no separate table)
    { "color": "silver", "price": 1299.99, "stock": 45 },
    { "color": "black",  "price": 1349.99, "stock": 12 }
  ],
  "tags": ["laptop", "business", "ultrabook"]
}

Single read returns the complete product — no joins.
Index on: sku (unique), category, tags (multikey index)
```

2.3 Wide-Column Stores

When to use: Time-series-style data per entity, write-heavy, known query patterns. **When NOT to use:** Ad-hoc queries, strong consistency, complex aggregations.

```
Cassandra data model design — query-first approach:

  Application query: "Show all events for user X, sorted by date desc, last 30"

  Table design (optimized for this query):
  CREATE TABLE user_events (
    user_id    UUID,
    event_date DATE,
    event_time TIMEUUID,
    event_type TEXT,
    payload    TEXT,
    PRIMARY KEY (user_id, event_date, event_time)
  ) WITH CLUSTERING ORDER BY (event_date DESC, event_time DESC);

  Partition key:    user_id  → determines which node stores the data
  Clustering cols:  event_date, event_time → determines sort within partition

  Query:
  SELECT * FROM user_events WHERE user_id = ? AND event_date >= ? LIMIT 30;
  ↑ Hits exactly 1 partition. No scatter-gather. O(log N) within partition.

  Anti-pattern (avoid):
  SELECT * FROM user_events WHERE event_type = 'purchase';
  ↑ Full cluster scan — Cassandra cannot index arbitrary columns efficiently
    without a pre-designed table or materialized view.

ScyllaDB vs Cassandra:
  Cassandra: JVM, shared thread pool, GC pauses cause latency spikes
  ScyllaDB:  C++, shard-per-core (each CPU core owns its own data partition)
             p99 latency: Cassandra ~50ms, ScyllaDB ~5ms (10x improvement)
             Drop-in Cassandra replacement with same CQL interface
```

2.4 Graph Databases

When to use: Relationships ARE the data (social, fraud, recommendations). **When NOT to use:** Simple entity lookup, bulk analytics, write-heavy workloads.

```
Relational vs Graph for "friends of friends" query:

  SQL (PostgreSQL):
  SELECT DISTINCT f2.friend_id
  FROM friendships f1
  JOIN friendships f2 ON f1.friend_id = f2.user_id
  WHERE f1.user_id = 42 AND f2.friend_id != 42;
```

```
  Depth 2: 1 join.  Depth 3: 2 joins.  Depth 6: 5 joins → exponential cost
  At 1M friendships: SQL query for depth-6 takes minutes.

  Neo4j (Cypher):
  MATCH (me:Person {id:42})-[:FRIEND*1..6]->(target:Person)
  WHERE target <> me
  RETURN DISTINCT target.name;

  Graph traversal: pointer-chasing through adjacency list
  At 1M friendships: depth-6 traversal takes milliseconds
  Why: No join computation — follows physical pointers to neighbor nodes

Named graph database systems:
  Neo4j:      Property graph, Cypher query language, ACID, single-node or cluster
  Amazon Neptune: Managed, supports both Property Graph (Gremlin) and RDF (SPARQL)
  TigerGraph: High-performance analytics graphs, parallel computation
  DGraph:     GraphQL native interface, distributed
```

---

3. SQL vs NoSQL Decision Framework

3.1 Full Comparison Matrix

Factor	PostgreSQL	MongoDB	Cassandra	Redis	Neo4j
ACID transactions	Full	Multi-doc (4.0+)	LWT only	Single-key	Full
Query language	SQL (mature)	MQL (good)	CQL (limited)	Commands	Cypher
Joins	Native, efficient	App-level	Not supported	Not supported	Native (graph)
Horizontal scale	Limited (Citus)	Native sharding	Native	Cluster	Limited
Write throughput	~50K/s	~100K/s	~1M/s	~1M/s	~10K/s
Schema flexibility	Rigid	Flexible	Semi-rigid	None	Flexible
Consistency	Strong	Configurable	Tunable	Strong (single)	Strong
Best write pattern	Mixed	Insert-heavy	Append-only	Any	Relationship-heavy

3.2 Decision Tree

```
START: What is your primary access pattern?

Is data relational with integrity constraints?
  YES → Do you need horizontal write scale?
          NO  → PostgreSQL (best default choice)
          YES → CockroachDB (distributed SQL) or Vitess (MySQL sharding)

Is data a graph (relationships are primary)?
  YES → Neo4j or Amazon Neptune

Is data time-ordered events, metrics?
  YES → InfluxDB, TimescaleDB, or Prometheus (see Section 4.5)

Is data primarily key-value lookup?
  YES → Is it a cache? → Redis
         Is it durable KV at scale? → DynamoDB

Is data document-shaped (nested, flexible schema)?
  YES → MongoDB
         Does it need extreme write scale? → Cassandra

Do you need analytics/aggregations at scale?
  YES → See Section 4.4 (Data Warehouses/Lakehouses)
```

---

4. Schema Topologies — When to Use Each

4.1 Schema Comparison

```
FLAT SCHEMA
  All attributes in one table. No joins.
  Example: simple_events(id, user_id, type, payload, ts)
  Use when: Single-entity access, no relationships, MVP/prototype
  Avoid when: Data changes frequently (update anomalies), relationships needed

NORMALIZED RELATIONAL (3NF)
  Multiple tables, foreign keys, referential integrity.
  Use when: OLTP (orders, accounts, inventory), data correctness critical
  Avoid when: High-read analytics, > 10 joins per query path

STAR SCHEMA
  Fact table (events/transactions) + Dimension tables (time, product, user,
location)

  orders_fact: { order_id, date_key, product_key, customer_key, amount, qty }
  date_dim:    { date_key, day, month, quarter, year, is_holiday }
  product_dim: { product_key, name, category, subcategory, brand }
```

```
  customer_dim:{ customer_key, name, region, tier, signup_date }

  Use when: Data warehouse, OLAP, BI dashboards
  Benefits: Simple queries (1 join per dimension), fast aggregations
  Avoid when: OLTP, frequent dimension updates

SNOWFLAKE SCHEMA
  Normalized dimensions (dimension tables have their own FK tables).
  product_dim → subcategory_dim → category_dim
  Use when: Storage is primary concern, dimensions updated frequently
  Avoid when: Query speed is priority (extra joins per dimension)

DOCUMENT (Embedded)
  All related data embedded in one document. No joins.
  Use when: Data is always read together, write-once-read-many, catalog
  Avoid when: Data shared across documents, frequent partial updates
```

---

5. Interview Use Cases & Design Problems

Problem 1 — E-Commerce Product Catalog: Document vs Relational

Problem statement: Design the database for an e-commerce catalog with 50M products across 200 categories. Each category has different attributes (electronics: RAM/CPU/GPU; clothing: size/color/material). You need to support full-text search, faceted filtering, and inventory tracking.

Solution:

```
Layer 1 — Product Catalog: MongoDB
  Reason: Each product document has different fields (flexible schema).
  Electronics: { sku, name, specs.ram_gb, specs.cpu, specs.gpu }
  Clothing:    { sku, name, size_options:["XS","S","M"], material }

  Index strategy:
    db.products.createIndex({ sku: 1 }, { unique: true })
    db.products.createIndex({ category: 1, price: 1 })
    db.products.createIndex({ tags: 1 })  // multikey index
    // Text search:
    db.products.createIndex({ name: "text", description: "text" })

Layer 2 — Inventory & Orders: PostgreSQL
  Reason: Stock levels require ACID (two people can't buy the last item).
  inventory: { sku TEXT PK, qty INT CHECK(qty >= 0), reserved INT }

  Atomic reservation:
    BEGIN;
    SELECT qty, reserved FROM inventory WHERE sku = 'X1' FOR UPDATE;
    -- Check available = qty - reserved >= requested
    UPDATE inventory SET reserved = reserved + 1 WHERE sku = 'X1';
```

```
    INSERT INTO orders ...;
    COMMIT;

Layer 3 — Search: Elasticsearch (or OpenSearch)
  Reason: Faceted filters + full-text + fuzzy matching not efficient in MongoDB.
  Sync via CDC (Change Data Capture) from MongoDB → Elasticsearch.

  Issues to address:
  - MongoDB → Elasticsearch sync lag (~1s): acceptable for catalog
  - Orphaned inventory when MongoDB product deleted: CDC delete event cleans up
  - Cross-system consistency: eventual (acceptable for product display)
```

When to use this approach: Product catalog with heterogeneous attributes AND strict inventory control. Use MongoDB for catalog flexibility, PostgreSQL for financial integrity, Elasticsearch for search.

Problem 2 — Social Network Feed: Graph vs Relational

Problem statement: Build a social feed for 50M users. Each user follows up to 5000 others. Show the 20 most recent posts from people a user follows. Query must complete in < 100ms.

Solution:

```
Approach A — Pull model (query at read time):

  PostgreSQL:
  SELECT p.* FROM posts p
  JOIN follows f ON p.user_id = f.followed_id
  WHERE f.follower_id = 42
  ORDER BY p.created_at DESC
  LIMIT 20;

  Problem: User follows 5000 people.
  At 100M posts: index on (user_id, created_at) still requires
  merging 5000 sorted streams → slow at scale.

Approach B — Push model (fanout on write):

  When Alice posts:
    1. Look up all of Alice's followers (could be 50M for celebrities!)
    2. Write post reference to each follower's feed table

  user_feeds: { user_id BIGINT, post_id BIGINT, ts TIMESTAMP }
  Index: (user_id, ts DESC)

  Query becomes:
  SELECT post_id FROM user_feeds WHERE user_id = 42 ORDER BY ts DESC LIMIT 20;
  ← Single index scan, O(log N + 20). Always < 10ms.
```

```
  Problem: Celebrity with 50M followers → 50M writes per post (fan-out storm)
  Solution: Hybrid — push for normal users (<10K followers), pull for celebrities

Implementation decision matrix:
  User follower count  | Strategy
  < 10K followers      | Push (fanout on write)
  > 10K followers      | Pull (merge at read time, cached)
  Celebrity post       | Lazy fanout: populate feed on first read, cache 5min

Database choice:
  Feed storage:  Cassandra (high write throughput, time-ordered per user)
  Follow graph:  Redis sorted set (fast follower lookups)
  Post content:  PostgreSQL (ACID, structured)
  Graph queries: Neo4j only if 2nd/3rd degree traversal needed
```

When to use graph DB here: Only if the product requires "posts from friends of friends" or "people you may know" suggestions — graph traversal 2-3 degrees deep. For simple follows: Redis sorted sets are faster and simpler.

Problem 3 — Multi-Tenant SaaS: Schema Strategy

Problem statement: Build a SaaS product for 10,000 business customers. Each customer has their own users, projects, and data. You need strict data isolation, the ability to customize schemas per customer, and to handle customers ranging from 1 user to 10,000 users.

Solution — Three schema isolation strategies:

```
Strategy 1 — Shared schema, row-level tenant isolation
  All customers in same tables, filtered by tenant_id.

  CREATE TABLE projects (
    id         BIGINT,
    tenant_id  UUID NOT NULL,  ← every table has this
    name       TEXT,
    ...
    PRIMARY KEY (tenant_id, id)  ← tenant_id first for partition pruning
  );

  PostgreSQL Row Level Security:
    CREATE POLICY tenant_isolation ON projects
    USING (tenant_id = current_setting('app.tenant_id')::UUID);
    ALTER TABLE projects ENABLE ROW LEVEL SECURITY;

  Pros: Simple operations, single schema to migrate
  Cons: One bad query can leak cross-tenant data, noisy neighbor on indexes
  Use when: Small tenants (<100 users), SaaS MVP, cost-sensitive

Strategy 2 — Separate schema per tenant (PostgreSQL schemas)
  CREATE SCHEMA tenant_acme;
```

```
  CREATE TABLE tenant_acme.projects (...);

  Pros: Namespace isolation, easy per-tenant backup/restore
  Cons: 10K schemas = pg_catalog bloat, migration complexity (10K ALTERs)
  Use when: 100–1000 tenants, regulatory compliance needed

Strategy 3 — Separate database per tenant
  Pros: Full isolation (separate connection pool, backups, failover)
  Cons: Expensive, connection pool explosion at 10K tenants
  Use when: Enterprise customers, contractual data residency requirements

RECOMMENDATION for 10K tenants ranging 1–10K users:
  Small tenants (< 100 users, ~9500 of them): shared schema + RLS
  Large tenants (> 100 users, ~500 of them):  separate schema
  Enterprise (> 1K users, ~50 of them):       separate database instance
  Migration path: start shared, promote tenants as they grow
```